GreenhouseOS 5.0 — Technical IP Deposit Document

Document Metadata

Property	Value
Document title	GreenhouseOS 5.0 — Intellectual Property Knowledge Vault
Document type	Technical architecture & originality reference
Software version	5.0.0
Compilation date	2026-09-09
Language	English
Classification	Confidential — IP deposit

Legal Notice

This document describes the architecture, algorithms, and design patterns of GreenhouseOS 5.0. It is prepared for intellectual property registration, technical due diligence, and internal knowledge management.

This document does not constitute a license grant. Source code remains governed by the BUSL-1.1 license file distributed with the software.

Purpose of This Deposit

Establish technical originality — Document proprietary integration of FAO-standard physics, real-time 3D twin rendering, and industrial climate-computer export in a unified SaaS platform.

Provide auditable architecture — Enable reviewers to verify system design without access to production infrastructure.

Support IP registration — Serve as a timestamped technical annex for copyright and trade-secret documentation workflows.

Redaction Statement

The following categories are intentionally excluded from this vault to protect company security and competitive position:

- API keys, tokens, passwords, and service-role credentials
- Production database URLs and connection strings
- Customer data, user accounts, and operational telemetry
- Full source code listings (architecture and algorithm descriptions only)
- Deployment-specific hostnames, IP addresses, and CDN configurations
- Unreleased roadmap features and pricing strategy

See 10-Security-Redaction-Policy for the complete exclusion list.

Table of Contents

Return to Home

GreenhouseOS 5.0 — Intellectual Property Knowledge Vault

Document classification: Internal IP deposit reference

Language: English

Version: 5.0.0

Generated for: Proprietary software registration and technical due diligence

Security posture: Redacted — no credentials, deployment secrets, or production endpoints

Navigation Map

This vault follows an Obsidian-style linked-note structure. Each note is self-contained yet cross-references related concepts via wikilinks.

Core Documents

Note	Purpose
00-IP-Deposit-Cover	Title page, legal notice, redaction policy
01-Executive-Summary	High-level product vision and value pillars
02-System-Overview	Monorepo topology and technology stack
03-Physics-Engine	FAO-56, VPD, DLI — agronomic calculation core
04-Thermal-Simulation	Energy balance, equipment models, heatmaps
05-3D-Frontend	React Three Fiber viewport, shaders, No-Form UX
06-AI-Gateway	Multi-provider AI copilot and local optimizer
07-Data-Persistence	Supabase schema, RLS, greenhouse CRUD
08-Industrial-Export	Priva / Ridder / Hoogendoorn interoperability
09-API-Contracts	REST and WebSocket public contracts (redacted)
10-Security-Redaction-Policy	What is excluded from this vault
11-Module-Index	Complete module registry with file paths
12-IP-Declaration	Ownership statement and originality claims

Concept Graph

```
graph TD
    A[Home] --> B[System Overview]
    B --> C[Physics Engine]
    B --> D[Thermal Simulation]
    B --> E[3D Frontend]
    B --> F[AI Gateway]
    B --> G[Data Persistence]
    C --> D
    D --> E
    F --> H[Industrial Export]
    G --> H
    E --> I[API Contracts]
    D --> I
```

Software Identity

Field	Value
Product name	GreenhouseOS 5.0

Category	Enterprise 3D Virtual Twin & SaaS for greenhouse design
License	Business Source License 1.1 (BUSL-1.1)
Change date	2029-08-03 → Apache 2.0
Source modules	113 TypeScript/Python files (~15,200 LOC)
Primary languages	Python 3.11+, TypeScript (strict)

Related External References

- FAO Irrigation and Drainage Paper 56 (Penman-Monteith ET_0)
- ASHRAE Handbook — Fundamentals (psychrometrics)
- Industrial climate computer tag conventions (Priva, Ridder, Hoogendoorn)

This vault is intended for IP deposit, investor technical annexes, and internal architecture review. It deliberately omits executable secrets. See 10-Security-Redaction-Policy.

Executive Summary

Home · 02-System-Overview

Product Vision

GreenhouseOS 5.0 is an enterprise-grade, open-source (BUSL-1.1) 3D Virtual Twin & SaaS platform for dynamic greenhouse design and agronomic simulation. Unlike conventional greenhouse CAD tools that treat 3D visualization as decorative, GreenhouseOS binds every viewport pixel to a physics probe — delivering actionable outcomes across four engineering pillars.

Four Value Pillars

1. Financial & Energy Impact (OPEX/CAPEX)

Output	Unit	Description
Energy consumption	kWh/m²	Real-time envelope and equipment load
Operational cost	€/day	Derived from thermal balance
CO_2 footprint	kg/day	Energy-to-emissions proxy
ROI	% / years	Payback on screens, LEDs, cooling upgrades

2. Advanced Agronomic Metrics

Metric	Unit	Standard
VPD	kPa	Magnus-Tetens psychrometrics
DLI	mol/m²/day	PAR-integrated daily light
Pathogen risk	index 0–1	Humidity × temperature stress proxy

3. Real Engineering & Sizing Outputs

Output	Unit
Heating boiler capacity	kW
Dehumidification rate	L/hour
Ventilation exchange	m³/h

4. Industrial Climate Computer Interop

Export of setpoint rules as standardized JSON payloads compatible with Priva, Ridder, and Hoogendoorn climate computers.

Development Milestones (Completed)

#	Milestone	Deliverable
1	Project setup & FAO physics	Penman-Monteith ET_0, VPD, DLI engines
2	3D canvas & i18n	React Three Fiber viewport, Zustand store, 4 locales
3	Thermodynamic core & WebSocket	Energy balance, <50 ms real-time loop
4	Interactive 3D & GLSL	TransformControls, heatmap shaders, InstancedMesh cr
5	Multi-AI gateway	OpenAI, Anthropic, Gemini, Ollama + local fallback
6	Supabase & launch polish	RLS schema, industrial dashboard, stress tests

Competitive Differentiation

No-Form UX — Design parameters edited directly in the 3D viewport via gizmos, not traditional forms.

Frontend–backend coefficient parity — Thermal solver constants synchronized across Python and TypeScript for consistent preview and server results.

Equipment-aware spatial heatmaps — Custom GLSL shaders visualize per-cell temperature and VPD influenced by pad walls, AC ducts, HAF fans, and vents.

Deterministic AI fallback — When cloud AI providers are unavailable, a rule-based local optimizer produces Priva/Ridder-compatible setpoints from FAO-56 metrics.

Technology Stack Summary

Layer	Technology
Backend	Python 3.11+, FastAPI, Pydantic v2
Frontend	React 18, TypeScript (strict), Vite
3D	Three.js, @react-three/fiber, @react-three/drei
State	Zustand
i18n	react-i18next (en, it, es, fr)
Database	Supabase (PostgreSQL + Row Level Security)
Real-time	WebSocket (/ws/simulation)
Container	Docker (backend)

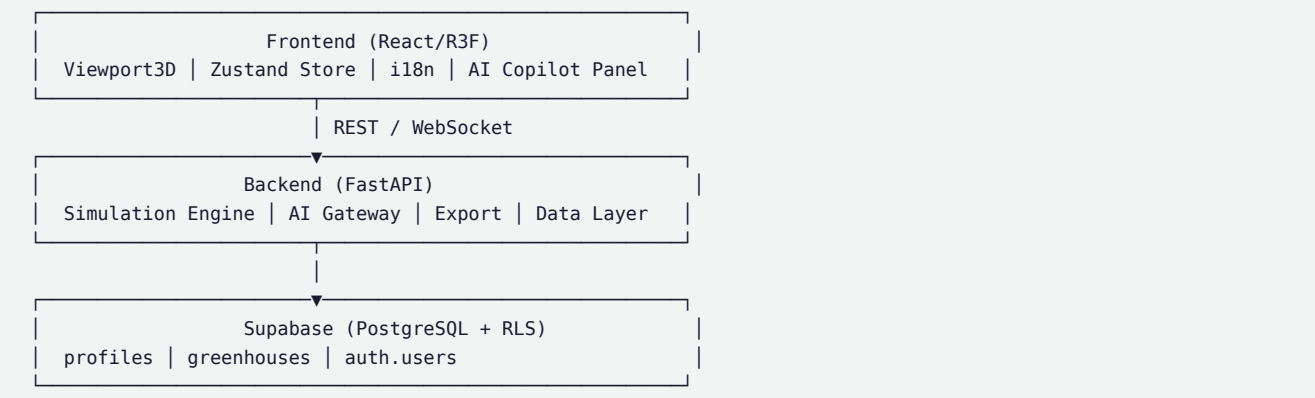
Continue to 02-System-Overview

System Overview

Home · 01-Executive-Summary · 03-Physics-Engine · 09-API-Contracts

Architectural Topology

GreenhouseOS follows a modular monorepo pattern with strict separation of concerns:



Repository Structure

```
greenhouse-os-5.0/
├── LICENSE                          # BUSL-1.1
├── README.md
├── CHANGELOG.md
├── docs/
│   ├── ARCHITECTURE.md
│   ├── API_SPEC.md
│   ├── THERMAL_EQUIPMENT_AUDIT.csv
│   └── ip-vault/                    # This knowledge vault
├── backend/
│   ├── app/
│   │   ├── main.py                  # FastAPI entrypoint
│   │   ├── core/                    # Configuration
│   │   ├── simulation/              # Physics engines
│   │   ├── ai/                      # Multi-AI gateway
│   │   ├── data/                    # Greenhouse CRUD
│   │   └── export/                  # Climate computer export
│   ├── requirements.txt
│   ├── Dockerfile
│   └── scripts/stress_test.py
├── frontend/
│   ├── src/
│   │   ├── components/3d/           # R3F viewport
│   │   ├── components/ui/           # HUD, dashboard
│   │   ├── components/ai/           # Copilot panel
│   │   ├── components/auth/         # Supabase auth
│   │   ├── hooks/                   # WebSocket, AI, auth
│   │   ├── store/                   # Zustand
│   │   ├── lib/thermal/             # Client-side thermal solver
│   │   └── locales/                 # en, it, es, fr
│   └── package.json
└── supabase/migrations/             # Database DDL
```

Backend Module Responsibilities

Module	Path	Responsibility

Entrypoint	backend/app/main.py	FastAPI app, CORS, route registration, WebSocket
Config	backend/app/core/config.py	Environment-based Pydantic Settings
Simulation	backend/app/simulation/	FAO-56, thermal, WebSocket handler
AI	backend/app/ai/	Multi-provider gateway, local optimizer
Data	backend/app/data/	Greenhouse persistence service
Export	backend/app/export/	Climate computer JSON generation

Frontend Module Responsibilities

Module	Path	Responsibility
3D Viewport	frontend/src/components/3d/	Canvas, mesh, shaders, equipment
UI Overlay	frontend/src/components/ui/	HUD, dashboard, controls
AI Copilot	frontend/src/components/ai/	Chat panel, GAIA integration
Auth	frontend/src/components/auth/	Supabase sign-in/up
State	frontend/src/store/useGreenhouseStore.ts	Single source of truth
Thermal lib	frontend/src/lib/thermal/	Client-side microclimate solver
Hooks	frontend/src/hooks/	WebSocket, AI, auth abstractions

Communication Patterns

REST API

- Prefix: /api/v1
- Simulation: POST /simulation/run — synchronous FAO-56 batch run
- AI: POST /ai/chat, POST /ai/optimize-climate
- Data: GET/POST/DELETE /greenhouses
- Export: POST /export/climate-computer
- Health: GET /health

WebSocket

- Endpoint: WS /ws/simulation
- Events: UPDATE_SIMULATION → SIMULATION_RESULTS, PING → PONG
- Target latency: p95 < 50 ms per update

See 09-API-Contracts for payload schemas.

Configuration Model

All runtime secrets are loaded from environment variables via Pydantic Settings. No credentials are embedded in source code. Configuration categories:

Category	Variables (names only)	Notes
Application	APP_NAME, DEBUG, CORS_ORIGINS	Non-sensitive defaults
AI providers	GEMINI_API_KEY	Optional; local optimizer fallback
Supabase	SUPABASE_URL, SUPABASE_ANON_KEY, SU	Required for production persistence

> Redacted: Actual values are never documented in this vault. See 10-Security-Redaction-Policy.

Deployment Topology (Abstract)

```
[Browser] ──HTTPS──► [CDN / Static Host] ──► React SPA
```

```
        │
        ├── REST/WS ──► [FastAPI Backend] ──► [Supabase PostgreSQL]
        │
        └── (optional) ──► [AI Provider APIs] via server-side proxy
```

Production hostnames, regions, and scaling parameters are deployment-specific and excluded from this document.

Continue to 03-Physics-Engine

Physics Engine — FAO-56 Agronomic Core

Overview

The agronomic physics engine implements FAO-56 Penman-Monteith reference evapotranspiration (ET_0) combined with Vapor Pressure Deficit (VPD) and Daily Light Integral (DLI) calculations. These form the biological foundation upon which the 04-Thermal-Simulation energy balance operates.

Primary implementations:

- backend/app/simulation/fao56.py
- backend/app/simulation/psychrometrics.py
- backend/app/simulation/vpd.py
- backend/app/simulation/dli.py
- backend/app/simulation/engine.py (orchestrator)

FAO-56 Penman-Monteith ET_0

Reference evapotranspiration for a well-watered grass reference crop:

$$
ET_0 = \frac{0.408 \cdot \Delta \cdot (R_n - G) + \gamma \cdot \frac{900}{T + 273} \cdot u_2 \cdot (e_s - e_a)}{\Delta + \gamma \cdot (1 + 0.34 \cdot u_2)}
$$

Symbol	Unit	Description
ET_0	mm/day	Reference evapotranspiration
R_n	MJ/m²/day	Net radiation at crop surface
G	MJ/m²/day	Soil heat flux (≈ 0 daily)
T	°C	Mean daily air temperature
u_2	m/s	Wind speed at 2 m height
e_s	kPa	Saturation vapor pressure
e_a	kPa	Actual vapor pressure
Δ	kPa/°C	Slope of saturation vapor pressure curve
γ	kPa/°C	Psychrometric constant

Pipeline Stages

1. Extraterrestrial radiation (R_a) — latitude and day-of-year dependent
2. Solar radiation (R_s) — Angstrom model or direct input
3. Net radiation (R_n) — albedo and longwave corrections
4. ET_0 — full Penman-Monteith assembly

Saturation Vapor Pressure (Magnus-Tetens)

$$
e_s = 0.6108 \cdot \exp\left(\frac{17.27 \cdot T}{T + 237.3}\right)
$$

Implementation: backend/app/simulation/psychrometrics.py

Vapor Pressure Deficit (VPD)

$$

VPD = e_s(T) - e_a

$$

VPD drives stomatal conductance and transpiration. Crop-stage optimal ranges:

Growth Stage	Optimal VPD (kPa)
Seedling	0.4 – 0.8
Vegetative	0.8 – 1.2
Generative	1.0 – 1.5

A VPD stress index (0–1) quantifies deviation from the optimal band for the selected crop and growth stage.

Implementation: backend/app/simulation/vpd.py

Daily Light Integral (DLI)

$$

DLI = 0.0864 \cdot f_{PAR} \cdot R_s \cdot \tau

$$

Symbol	Unit	Description
DLI	mol/m²/day	Daily light integral
f_{PAR}	mol/MJ	PAR fraction (2.08)
R_s	MJ/m²/day	Global solar radiation
τ	—	Covering transmittance (0–1)

A DLI adequacy index compares calculated DLI against crop-specific requirements.

Implementation: backend/app/simulation/dli.py

Supported Crops & Growth Stages

Crop Types

tomato, cucumber, pepper, lettuce, strawberry, cannabis

Growth Stages

seedling, early_vegetative, mid_season, late_vegetative, generative, harvest

Each crop/stage combination carries calibrated Kc (crop coefficient), VPD band, and DLI target values defined in backend/app/simulation/constants.py and mirrored in frontend cultivation factors.

Simulation Engine Orchestration

```
SimulationRequest (JSON)
    │
    ▼
SimulationEngine.run()
    ├── calculate_et0()        → ET0Result
    ├── calculate_vpd_kpa()  → VPD + stress index
    └── calculate_dli()      → DLI + adequacy index
    │
    ▼
SimulationResponse (JSON)
```

Entry point: POST /api/v1/simulation/run

Schema validation: Pydantic v2 models in backend/app/simulation/schemas.py

Physical Constants

Centralized in backend/app/simulation/constants.py:

- Psychrometric constant γ
- PAR fraction f_{PAR}
- Crop coefficients (Kc) per crop/stage
- Albedo values for net radiation
- ASHRAE-aligned psychrometric helpers

References

- Allen, R.G., Pereira, L.S., Raes, D., Smith, M. (1998). Crop Evapotranspiration — Guidelines for Computing Crop Water Requirements. FAO Irrigation and Drainage Paper 56.
- ASHRAE Handbook — Fundamentals (psychrometric properties).

Continue to 04-Thermal-Simulation

Thermal Simulation — Energy Balance & Equipment Models

Home · 03-Physics-Engine · 05-3D-Frontend

Overview

The thermal simulation layer solves a quasi-steady-state greenhouse energy balance in real time, coupling FAO-56 transpiration with envelope conduction, ventilation, and climate equipment loads. Frontend and backend implementations share calibrated coefficients for parity.

Primary implementations:

- backend/app/simulation/thermal_physics.py
- backend/app/simulation/thermal.py
- backend/app/simulation/realtime_engine.py
- backend/app/simulation/climate_equipment.py
- frontend/src/lib/thermal/solveMicroclimate.ts
- frontend/src/lib/thermalEstimate.ts

Equipment specification audit: docs/THERMAL_EQUIPMENT_AUDIT.csv

Greenhouse Energy Balance

$$
Q_{net} = Q_{solar} + Q_{transpiration} + Q_{ventilation} + Q_{conduction} \approx 0
$$

Flux	Unit	Description
Q_{solar}	W/m²	Transmitted solar gain through covering
$Q_{transpiration}$	W/m²	Latent heat from crop ET (FAO-56 × Kc × LAI)
$Q_{ventilation}$	W/m²	Sensible heat loss via air exchange (ACH)
$Q_{conduction}$	W/m²	Envelope heat loss via U-value

Internal temperature is solved at quasi-steady state from net energy gain and total conductance.

Key Physical Coefficients

Parameter	Value	Notes
Air density ρ	1.2 kg/m³	Ventilation sensible heat
Air c_p	1005 J/(kg·K)	Ventilation sensible heat
Latent heat λ	2.45 MJ/kg	Transpiration latent heat
Solar absorption	0.72 × τ	Fraction of transmitted radiation heating air
RH coupling	1.0 %/°C	Internal RH shift per °C below external dry-bulb
Cooling capacity factor	1.15	Applied to pad, fog, evaporative, and AC deltas

Ventilation Model (Qatar-Corrected)

Flow-based ventilation combining:

Mechanism	Description
Mechanical	Exhaust fan throat area × velocity
Wind	Facade pressure differential

Stack	Buoyancy-driven ridge vent flow
Infiltration	Baseline air leakage (ACH)

ACH fan boost: $(A_{fan}/A_{floor}) \times 8$

ACH vent boost: $(A_{vent}/A_{floor}) \times 2.5$

ACH circulation: $\min(N_{circ}, 24) \times 0.15$ (HAF mixing only — no direct exhaust)

Climate Equipment Models

Equipment	Model Behavior
Fan-and-pad	Wet-bulb depression with diminishing returns; pad outlet sta
Evaporative cooling	$T_{wb} - 1.5/1.15$ °C floor
High-pressure fog	Full-length RH bands aloft
Mechanical AC	kW-rated with high-ambient derating; 12 °C floor
Heating	Warm spots near heaters; gated below 22 °C setpoint
HAF circulation	Mixing only — dampens local ΔT/ΔRH, no cooling
Shading screen	Solar flux attenuation

Rated capacity fields (m³/h, kW, COP, SHR, pad/fog efficiency) are exposed in the UI and wired through WebSocket payloads.

Moisture Balance

Humidity-ratio based moisture balance tracks:

- Transpiration moisture addition
- Ventilation moisture exchange
- Pad/fog evaporative humidification
- Per-cell RH derivation for spatial heatmaps

Spatial Heatmap Engine

The heatmap produces a 2D matrix of per-cell temperature and VPD values influenced by equipment placement:

System	Spatial Pattern
Solar	Directional beam on roof/walls/floor
Pad / evaporative	Cool humid plume at pad wall + airflow transit
Exhaust fans	Cool/dry near exhaust gable
HAF circulation	Mixing (dampens local ΔT/ΔRH)
Roof/side vents	Cool near openings
Mechanical AC	Duct network + diffusers
High-pressure fog	Full-length bands along fog lines
Heating	Warm spots near heaters

Frontend: frontend/src/lib/equipmentAwareHeatmap.ts, frontend/src/lib/thermal/spatialField.ts

Shader rendering: See 05-3D-Frontend#GLSL Heatmap Shader

Heatmap color scale: fixed 0–50 °C reference with crop working-range band.

Real-Time WebSocket Pipeline

```
  Client                          Server
    │                               │
    ├── UPDATE_SIMULATION ─────────►│ RealtimeSimulationEngine.run()
```

```
    │                                │   ├── FAO-56 ET0 (~0.3 ms)
    │                                │   └── Thermal balance (~0.4 ms)
    │◄── SIMULATION_RESULTS ─────────┤
    │                                │
    ├── PING ──────────────────────►│
    │◄── PONG ──────────────────────┤
```

Endpoint: WS /ws/simulation

Handler: backend/app/simulation/websocket_handler.py

Frontend hook: frontend/src/hooks/useSimulationWS.ts

Target: p95 < 50 ms per update (validated by backend/scripts/stress_test.py)

Cultivation Thermal Mass

A cultivation thermal mass divisor (≥ 1.0) dampens temperature deviation from steady state without sub-unity amplification, modeling the buffering effect of substrate and crop biomass.

Continue to 05-3D-Frontend

3D Frontend — Virtual Twin & No-Form UX

Overview

The frontend delivers a No-Form UX paradigm: greenhouse design parameters are edited directly in a 3D viewport via gizmos and sliders, with immediate visual and physics feedback. Built on React Three Fiber (R3F) atop Three.js.

Primary implementations:

- frontend/src/components/3d/Viewport3D.tsx
- frontend/src/components/3d/GreenhouseScene.tsx
- frontend/src/components/3d/GreenhouseMesh.tsx
- frontend/src/store/useGreenhouseStore.ts

State Management (Zustand)

The useGreenhouseStore holds all greenhouse design parameters as a single reactive source of truth:

Slice	Fields	Derived
Geometry	length, width, ridgeHeight, eaveHeight	floorAreaM2, volumeM3, ridgeAngleDeg
Covering	type, transmittance, uValue	Glass opacity in 3D mesh
Crop	type, system, lai, growthStage	HUD labels via i18n
Location	lat, lon, elevationM	HUD coordinates
Climate equipment	fans, vents, pads, AC, fog, heating	Layout positions, rated capacities
Simulation results	thermal balance, heatmap matrix	Shader uniforms, HUD metrics

Dimension changes trigger React re-renders, rebuilding roof geometry via useMemo and updating wall/roof scales.

3D Rendering Pipeline

```
Viewport3D (Canvas)
    ├── OrbitControls (damped camera)
    ├── Grid (infinite ground plane)
    ├── GreenhouseMesh
    │       ├── Glass walls (meshPhysicalMaterial + transmission)
    │       ├── Gable roof (custom BufferGeometry)
    │       └── Structural frame (ridge beam + arch ribs)
    ├── CropGridMesh (InstancedMesh foliage)
    ├── ClimateEquipmentMesh (fans, pads, ducts, heaters)
    └── HeatmapShader (floor temperature/VPD overlay)
```

No-Form TransformControls

Direct in-viewport geometry editing via @react-three/drei TransformControls:

Mode	Action
Scale	Maps X/Y/Z scale to length/ridge/eave height and width
Translate	Reposition (resets to origin on release)
Off	Standard orbit navigation only

OrbitControls are disabled during gizmo drag to prevent camera conflict.

Implementation: frontend/src/components/3d/GreenhouseScene.tsx

GLSL Heatmap Shader

Custom ShaderMaterial renders simulation heatmap_matrix as a floor overlay:

- Temperature mode — Blue → green → yellow → red gradient (0–50 °C)
- VPD mode — Optimal green → stress orange → severe red

Data uploaded via THREE.DataTexture (Float32, RedFormat).

Files:

- frontend/src/components/3d/HeatmapShader.tsx
- frontend/src/components/3d/shaders/heatmapShader.ts
- frontend/src/lib/heatmapFallback.ts
- frontend/src/lib/heatmapRevision.ts

InstancedMesh Crop Grid

High-performance foliage rendering using THREE.InstancedMesh:

- Grid spacing per crop type (lettuce 0.25 m, tomato 0.5 m, etc.)
- Instance scale from LAI × growth stage
- Crop-specific color palette

Implementation: frontend/src/components/3d/CropGridMesh.tsx, frontend/src/lib/plantGeometry.ts

Mechanical AC Duct Network

When cooling mode is mechanical_ac, each wall-mounted unit drives an internal supply network:

Element	Rule
Trunk	Short tap from each AC unit to its wall header
Headers	Full greenhouse length along north and/or south wall
Cross ducts	Every ~5.5 m, span full width
Diffusers	Every 4 m on headers and along cross ducts
Diameter	Scales with ac_unit_width_m (0.18–0.24 m)

Layout: frontend/src/lib/acDuctLayout.ts

3D mesh: frontend/src/components/3d/ClimateEquipmentMesh.tsx

UI Components

Component	Path	Function
HUDOverlay	components/ui/HUDOverlay.tsx	Real-time metrics overlay
ClimateDashboard	components/ui/ClimateDashboard.tsx	OPEX, energy, CO_2, export
DimensionControls	components/ui/DimensionControls.tsx	Structure sliders
CultivationClimateControls	components/ui/CultivationClimateControls.	Crop & equipment controls
HeatmapControls	components/ui/HeatmapControls.tsx	Heatmap mode toggle
LanguagePicker	components/ui/LanguagePicker.tsx	Locale selector
GizmoToolbar	components/ui/GizmoToolbar.tsx	Transform mode selector

Internationalization (i18n)

Locale	Code	Namespaces
English	en	common, simulation, crops, 3d_controls, ai_copilot
Italian	it	common, simulation, crops, 3d_controls, ai_copilot
Spanish	es	common, simulation, crops, 3d_controls, ai_copilot

French	fr	common, simulation, crops, 3d_controls, ai_copilot

Locale selection syncs between the Zustand store and i18next.changeLanguage().

Implementation: frontend/src/i18n.ts, frontend/src/locales/

Client-Side Thermal Preview

The frontend runs a parallel thermal solver (frontend/src/lib/thermal/) for instant preview without WebSocket round-trip:

- solveMicroclimate.ts — steady-state solver
- ventilationFlow.ts — flow-based ventilation
- equipmentLoads.ts — equipment heat/cool loads
- spatialField.ts — per-cell heatmap generation
- ratedCapacities.ts — equipment capacity resolution

Coefficients are synchronized with the backend per 04-Thermal-Simulation.

Continue to 06-AI-Gateway

AI Gateway — Multi-Provider Copilot

Home · 02-System-Overview · 08-Industrial-Export

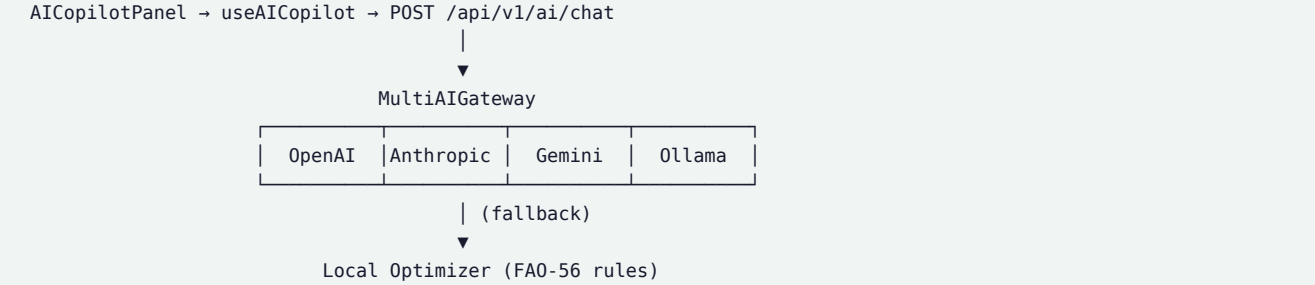
Overview

GreenhouseOS includes a decoupled Multi-AI gateway that routes natural-language climate optimization requests to cloud AI providers, with a deterministic local fallback when providers are unavailable.

Primary implementations:

- backend/app/ai/gateway.py
- backend/app/ai/base.py
- backend/app/ai/local_optimizer.py
- backend/app/ai/router.py
- frontend/src/components/ai/AICopilotPanel.tsx
- frontend/src/hooks/useAICopilot.ts

Architecture



Provider Abstraction

Each provider implements the abstract AIProvider.complete() interface:

Provider	Implementation	Transport
OpenAI	providers/openai_provider.py	HTTPS REST
Anthropic	providers/anthropic_provider.py	HTTPS REST
Gemini	providers/gemini_provider.py	HTTPS REST
Ollama	providers/ollama_provider.py	Local HTTP

Provider availability is determined at runtime by checking whether the corresponding API key or endpoint is configured. No API keys are stored in source code.

Local Optimizer Fallback

When the selected cloud provider is unavailable or fails, local_optimizer.py produces deterministic, rule-based setpoints derived from:

- Current VPD and temperature readings
- Crop type and growth stage
- FAO-56 stress indices

Output format is compatible with 08-Industrial-Export climate computer payloads (Priva/Ridder tag conventions).

GAIA Frontend Integration

The frontend includes a GAIA (Generative AI Assistant) layer for in-browser copilot chat:

File	Role
frontend/src/lib/gaia/client.ts	API client
frontend/src/lib/gaia/buildContext.ts	Simulation context assembly
frontend/src/lib/gaia/formatContext.ts	Context formatting for prompts
frontend/src/lib/gaia/prompts.ts	System prompt templates
frontend/src/components/ai/GaiaMarkdown.tsx	Rendered response display
frontend/api/gaia.ts	Serverless proxy endpoint (Vercel)

API keys for GAIA are resolved at runtime from environment variables. Variable names are documented in 10-Security-Redaction-Policy; values are never included in this vault.

REST Endpoints

Method	Path	Description
GET	/api/v1/ai/providers	List available providers and status
POST	/api/v1/ai/chat	Natural-language climate chat
POST	/api/v1/ai/optimize-climate	Structured setpoint optimization

Request/response schemas: backend/app/ai/schemas.py

Prompt Engineering

System prompts in backend/app/ai/prompts.py constrain AI responses to:

- Agronomic best practices aligned with FAO standards
- Actionable setpoint recommendations (temperature, RH, ventilation)
- Industrial climate computer tag format awareness
- Refusal to disclose system credentials or internal configuration

Security Considerations

- AI provider API keys are server-side only (backend env or serverless proxy)
- Frontend never receives or stores provider credentials
- Chat context includes only simulation parameters — no user PII beyond optional greenhouse name
- See 10-Security-Redaction-Policy

Continue to 07-Data-Persistence

Data Persistence — Supabase & Greenhouse CRUD

Overview

GreenhouseOS persists user profiles and greenhouse designs via Supabase (PostgreSQL with Row Level Security). The backend provides a CRUD API with an in-memory fallback for local development when Supabase is not configured.

Primary implementations:

- supabase/migrations/001_initial_schema.sql
- backend/app/data/greenhouse_service.py
- backend/app/data/router.py
- frontend/src/lib/supabase.ts
- frontend/src/hooks/useAuth.ts
- frontend/src/components/auth/AuthPanel.tsx

Database Schema

```
auth.users ──► profiles (RLS: auth.uid() = id)
                  │
                  └──► greenhouses (RLS: auth.uid() = user_id)
                            └── public SELECT when is_public = TRUE
```

Table: profiles

Column	Type	Description
id	UUID (PK, FK → auth.users)	User identity
full_name	TEXT	Display name
company_name	TEXT	Organization
preferred_language	VARCHAR(5)	Default locale (en, it, es, fr)
ai_provider_preferences	JSONB	Default AI provider/model selection
created_at	TIMESTAMPTZ	Record creation
updated_at	TIMESTAMPTZ	Auto-updated via trigger

Table: greenhouses

Column	Type	Description
id	UUID (PK)	Greenhouse identity
user_id	UUID (FK → profiles)	Owner
name	TEXT	Design name
description	TEXT	Optional notes
latitude	DOUBLE PRECISION	Geolocation
longitude	DOUBLE PRECISION	Geolocation
dimensions	JSONB	{length, width, ridge_height, eave_height}
covering_material	JSONB	{type, transmittance, u_value}
crop_config	JSONB	{crop_type, cultivation_system, lai, growth_stage}
is_public	BOOLEAN	Public visibility flag
created_at	TIMESTAMPTZ	Record creation
updated_at	TIMESTAMPTZ	Auto-updated via trigger

Row Level Security (RLS)

Policy	Table	Rule
Users can manage their profile	profiles	auth.uid() = id
Users can manage their greenhouses	greenhouses	auth.uid() = user_id
Public greenhouses accessible by all	greenhouses	is_public = TRUE (SELECT only)

RLS ensures users can only read/write their own data unless a greenhouse is explicitly marked public.

Authentication Flow

```
AuthPanel (Supabase Auth) → user.id
    │
    ▼
ClimateDashboard "Save" → POST /api/v1/greenhouses (X-User-Id header)
    │
    ├── Supabase PostgREST (production)
    └── In-memory dict (local dev fallback)
```

- Sign-in/sign-up via supabase.auth.signInWithPassword / signUp
- Graceful degradation when Supabase is not configured (auth panel shows unconfigured state)
- No passwords or tokens are logged or stored in application code

CRUD API

Method	Path	Description
GET	/api/v1/greenhouses	List user's greenhouses
POST	/api/v1/greenhouses	Create or update greenhouse design
DELETE	/api/v1/greenhouses/{id}	Delete greenhouse

Authentication is via the X-User-Id header (Supabase user UUID) in the backend service layer. Production deployments should validate JWT tokens at the API gateway level.

TypeScript Types

Frontend Supabase types are defined in frontend/src/types/supabase.ts, generated to match the migration schema. Greenhouse domain types are in frontend/src/types/greenhouse.ts.

Security Notes

- SUPABASE_SERVICE_ROLE_KEY is used only server-side for admin operations
- SUPABASE_ANON_KEY is safe for client-side use (RLS enforces access)
- Production URLs and keys are never documented in this vault
- See 10-Security-Redaction-Policy

Continue to 08-Industrial-Export

Industrial Export — Climate Computer Interoperability

Home · 06-AI-Gateway · 07-Data-Persistence

Overview

GreenhouseOS exports simulation-derived setpoint rules as standardized JSON payloads compatible with industrial climate computers from Priva, Ridder, and Hoogendoorn.

Primary implementations:

- backend/app/export/climate_computer.py
- backend/app/export/schemas.py
- backend/app/export/router.py
- frontend/src/lib/climateExport.ts

Supported Formats

Vendor	Tag Prefix Convention	Output
Priva	Priva. namespace	Temperature, RH, ventilation setpoints
Ridder	Ridder. namespace	Climate control rules
Hoogendoorn	HGC. namespace	Process computer setpoints

Format selection is a request parameter; the export engine applies vendor-specific tag prefixes and unit conventions.

Export Content

Each export payload includes:

Category	Fields
Temperature setpoints	Day/night targets, ramp rates
Humidity setpoints	RH targets per crop growth stage
Ventilation rules	Min/max vent positions, CO_2 override thresholds
Screen control	Thermal screen open/close conditions
Heating	Boiler enable thresholds
Cooling	Pad/AC activation conditions

Setpoints are derived from:

- Current simulation microclimate (VPD, temperature, DLI)
- Crop type and growth stage optimal bands
- AI copilot recommendations (when available)
- Local optimizer fallback rules

REST Endpoint

Method	Path	Description
POST	/api/v1/export/climate-computer	Generate vendor-specific JSON

Request schema includes:

- format: priva | ridder | hoogendoorn
- greenhouse_id (optional): load saved design

- Inline simulation parameters (alternative to saved design)

Response: vendor-formatted JSON object ready for import into climate computer configuration tools.

Frontend Integration

The ClimateDashboard component (frontend/src/components/ui/ClimateDashboard.tsx) displays:

- OPEX estimate (€/day)
- Energy consumption (kWh/m²)
- CO_2 footprint (kg/day)
- Export button with format selector

Export is triggered client-side via frontend/src/lib/climateExport.ts, calling the backend export endpoint.

Industrial Dashboard Metrics

Metric	Calculation Basis
OPEX €/day	Equipment kW loads × energy tariff proxy
Energy kWh/m²	Sum of heating, cooling, fan, lighting loads / floor area
CO_2 kg/day	Energy × emissions factor

These are engineering estimates for design-phase decision support, not operational billing data.

Continue to 09-API-Contracts

API Contracts — REST & WebSocket (Redacted)

Base URL

```
http://localhost:8000          # Local development
```

Production base URLs are deployment-specific and excluded from this document.

Health Check

GET /health

Response 200:

```
{
  "status": "healthy",
  "version": "5.0.0"
}
```

Simulation

POST /api/v1/simulation/run

Execute FAO-56 Penman-Monteith ET_0 with VPD and DLI agronomic metrics.

Request (representative):

```
{
  "climate": {
    "latitude_deg": 41.9028,
    "longitude_deg": 12.4964,
    "temperature_max_c": 34.0,
    "temperature_min_c": 22.0,
    "relative_humidity_pct": 72.0,
    "wind_speed_m_s": 2.5
  },
  "covering": { "type": "glass", "transmittance": 0.85, "u_value": 5.8 },
  "crop_type": "tomato",
  "growth_stage": "mid_season"
}
```

Response (representative):

```
{
  "et0": {
    "et0_mm_day": 4.852,
    "net_radiation_mj_m2_day": 12.340,
    "solar_radiation_mj_m2_day": 22.150,
    "daylight_hours": 14.52
  },
  "agronomic": {
    "vpd_kpa": 1.254,
    "vpd_stress_index": 0.045,
    "dli_mol_m2_day": 18.732,
    "dli_adequacy_index": 0.937
  }
}
```

Full schema: backend/app/simulation/schemas.py

WebSocket — Real-Time Simulation

Endpoint

```
WS /ws/simulation
```

Client → Server: UPDATE_SIMULATION

```
{
  "event": "UPDATE_SIMULATION",
  "data": {
    "location": { "lat": 41.9028, "lon": 12.4964 },
    "geometry": { "length": 30.0, "width": 10.0, "ridge_height": 4.5, "eave_height": 3.0 },
    "materials": { "covering_type": "glass", "transmittance": 0.85, "u_value": 5.8 },
    "crop": { "type": "tomato", "system": "hydroponic_nft", "lai": 3.2, "growth_stage": "mid_season" }
  }
}
```

Server → Client: SIMULATION_RESULTS

```
{
  "event": "SIMULATION_RESULTS",
  "data": {
    "thermal_balance": {
      "q_solar": 450.5,
      "q_transpiration": -120.3,
      "q_ventilation": -80.2,
      "q_net_delta": 250.0
    },
    "microclimate": {
      "internal_temp": 28.4,
      "external_temp": 34.0,
      "internal_rh": 72.1,
      "vpd_kpa": 1.25,
      "et0_fao56": 4.85
    },
    "heatmap_matrix": ["..."]
  }
}
```

Heartbeat

```
{ "event": "PING" }  →  { "event": "PONG" }
```

Schema: backend/app/simulation/websocket_schemas.py

AI Endpoints

Method	Path	Description
GET	/api/v1/ai/providers	List providers and availability
POST	/api/v1/ai/chat	Natural-language climate chat
POST	/api/v1/ai/optimize-climate	Structured optimization

Data Endpoints

Method	Path	Description
GET	/api/v1/greenhouses	List user greenhouses
POST	/api/v1/greenhouses	Create/update greenhouse

DELETE	/api/v1/greenhouses/{id}	Delete greenhouse

Export Endpoints

Method	Path	Description
POST	/api/v1/export/climate-computer	Generate vendor JSON

CORS Policy

- Development origins: localhost:5173, localhost:3000
- Production: regex pattern for *.vercel.app deployments
- Credentials allowed; all methods and headers permitted

Actual production origin lists are environment-configured.

Error Responses

Standard FastAPI error format:

```
{
  "detail": "Validation error description"
}
```

HTTP status codes: 200 (success), 400 (validation), 401 (auth), 404 (not found), 422 (schema), 500 (server).

Continue to 10-Security-Redaction-Policy

Security & Redaction Policy

Home · 00-IP-Deposit-Cover

Purpose

This policy defines what information is included and excluded from the GreenhouseOS IP Knowledge Vault to ensure the document can be safely deposited, shared with IP attorneys, or submitted to registration authorities without creating security vulnerabilities or disclosing competitive secrets.

Included (Safe for IP Deposit)

Category	Examples
Architecture diagrams	System topology, data flow, module relationships
Algorithm descriptions	FAO-56 formulas, energy balance equations, VPD/DLI math
Module registry	File paths, responsibilities, line counts
API contracts	Endpoint paths, request/response shapes (no auth tokens)
Database schema	Table structures, column types, RLS policies
Technology stack	Framework names, language versions
Milestone history	Feature delivery timeline
License terms	BUSL-1.1 summary and change date
Physical constants	Published engineering coefficients
i18n structure	Locale codes and namespace names

Excluded (Never in This Vault)

Credentials & Secrets

Item	Reason
API keys (Gemini, OpenAI, Anthropic)	Authentication bypass risk
Supabase service role key	Full database access
Supabase anon key (production)	Tied to production project
JWT tokens, session cookies	Session hijacking
Database passwords	Direct data access
Docker registry credentials	Supply chain risk

Environment variable names (e.g., GEMINI_API_KEY, SUPABASE_URL) may be referenced; values are never documented.

Infrastructure Details

Item	Reason
Production hostnames / URLs	Attack surface enumeration
Server IP addresses	Direct targeting
CDN configurations	Infrastructure mapping
SSL certificate details	Certificate pinning bypass
Load balancer rules	Traffic manipulation
Container orchestration secrets	Cluster compromise

Business-Sensitive Data

Item	Reason
Customer names, emails, accounts	GDPR / privacy
Pricing strategy	Competitive intelligence
Unreleased roadmap	Market positioning
Partnership agreements	Legal confidentiality
Revenue figures	Financial disclosure

Full Source Code

Item	Reason
Complete file contents	Enables unauthorized reproduction
Proprietary algorithm implementations	Trade secret protection

This vault describes what the software does and how it is architected, not the full executable source. Source remains under BUSL-1.1 license control.

Redaction Verification Checklist

Before distributing this document, verify:

- [ ] No .env file contents are included
- [ ] No API keys or tokens appear anywhere
- [ ] No production URLs or IP addresses
- [ ] No customer or user data
- [ ] No full source code listings
- [ ] Example JSON uses fictional/generic coordinates only
- [ ] Environment variable names only, never values

Safe Example Data

All example coordinates, temperatures, and simulation parameters in this vault use generic demonstration values (e.g., Rome coordinates 41.9028°N, 12.4964°E) that do not correspond to any real customer installation.

Incident Response

If this vault is found to contain inadvertently included secrets:

1. Immediately rotate all exposed credentials
2. Regenerate the vault with updated redaction
3. Revoke previously distributed copies
4. Audit access logs for the affected services

Continue to 11-Module-Index

Module Index — Complete File Registry

Home · 02-System-Overview · 12-IP-Declaration

Summary Statistics

Metric	Value
Total source files	113
Total lines of code	~15,231
Backend Python modules	~45
Frontend TypeScript/TSX modules	~68
SQL migrations	1
Documentation files	5+
Supported locales	4 (en, it, es, fr)

Backend Modules

Core

File	Lines (approx.)	Responsibility
backend/app/main.py	57	FastAPI entrypoint, CORS, routes
backend/app/core/config.py	36	Pydantic Settings from env

Simulation Engine

File	Responsibility
simulation/engine.py	SimulationEngine orchestrator
simulation/fao56.py	Penman-Monteith ET_0 pipeline
simulation/psychrometrics.py	Magnus-Tetens, Δ, γ
simulation/vpd.py	VPD calculation + stress index
simulation/dli.py	DLI + adequacy index
simulation/thermal.py	Thermal balance orchestration
simulation/thermal_physics.py	Energy balance coefficients
simulation/realtime_engine.py	WebSocket real-time solver
simulation/climate_equipment.py	Equipment load models
simulation/cultivation.py	Crop thermal mass, Kc
simulation/geometry.py	Greenhouse geometry helpers
simulation/shading_screen.py	Screen solar attenuation
simulation/constants.py	Physical constants
simulation/schemas.py	Pydantic request/response models
simulation/websocket_handler.py	WebSocket connection manager
simulation/websocket_schemas.py	WS event schemas

AI Gateway

File	Responsibility
ai/gateway.py	MultiAIGateway router
ai/base.py	AIProvider abstract interface
ai/local_optimizer.py	Deterministic fallback optimizer
ai/router.py	FastAPI AI routes

ai/schemas.py	AI request/response models
ai/prompts.py	System prompt templates
ai/providers/gemini_provider.py	Google Gemini integration
ai/providers/__init__.py	Provider registry

Data Layer

File	Responsibility
data/greenhouse_service.py	Supabase CRUD + in-memory fallback
data/router.py	Greenhouse REST endpoints
data/schemas.py	Data transfer models

Export

File	Responsibility
export/climate_computer.py	Vendor JSON generation
export/schemas.py	Export request/response models
export/router.py	Export REST endpoints

Scripts

File	Responsibility
scripts/stress_test.py	REST + WS latency validation

Frontend Modules

3D Components

File	Responsibility
components/3d/Viewport3D.tsx	R3F Canvas setup
components/3d/GreenhouseScene.tsx	Scene graph + TransformControls
components/3d/GreenhouseMesh.tsx	Parametric greenhouse geometry
components/3d/CropGridMesh.tsx	InstancedMesh crop foliage
components/3d/ClimateEquipmentMesh.tsx	Fans, pads, ducts, heaters
components/3d/HeatmapShader.tsx	GLSL heatmap overlay
components/3d/shaders/heatmapShader.ts	Vertex/fragment shaders

UI Components

File	Responsibility
components/ui/HUDOverlay.tsx	Real-time metrics HUD
components/ui/ClimateDashboard.tsx	OPEX, energy, export
components/ui/DimensionControls.tsx	Structure dimension sliders
components/ui/CultivationClimateControls.tsx	Crop & equipment controls
components/ui/HeatmapControls.tsx	Heatmap mode selector
components/ui/HeatmapScaleLegend.tsx	Color scale legend
components/ui/LanguagePicker.tsx	Locale selector
components/ui/GizmoToolbar.tsx	Transform mode toolbar
components/ui/RangeGauge.tsx	Metric gauge widget
components/ui/StatusBadge.tsx	Connection status indicator

AI Components

File	Responsibility
components/ai/AICopilotPanel.tsx	Chat UI
components/ai/GaiaMarkdown.tsx	Markdown response renderer
components/ai/GaiaSiteContext.tsx	GAIA context provider

Auth

File	Responsibility
components/auth/AuthPanel.tsx	Sign-in/sign-up form

Hooks

File	Responsibility
hooks/useSimulationWS.ts	WebSocket connection + auto-reconnect
hooks/useAICopilot.ts	AI chat state management
hooks/useAuth.ts	Supabase auth state

State & Types

File	Responsibility
store/useGreenhouseStore.ts	Zustand global state
types/greenhouse.ts	Greenhouse domain types
types/simulation.ts	Simulation result types
types/ai.ts	AI provider types
types/supabase.ts	Database types
types/viewport.ts	3D viewport types

Thermal Library

File	Responsibility
lib/thermal/solveMicroclimate.ts	Client steady-state solver
lib/thermal/ventilationFlow.ts	Flow-based ventilation
lib/thermal/equipmentLoads.ts	Equipment heat/cool loads
lib/thermal/spatialField.ts	Per-cell heatmap field
lib/thermal/ratedCapacities.ts	Equipment capacity resolution
lib/thermal/constants.ts	Frontend thermal constants
lib/thermal/vpd.ts	Client VPD calculation
lib/thermal/psychrometricsExtended.ts	Extended psychrometrics

Layout & Utility Libraries

File	Responsibility
lib/acDuctLayout.ts	AC duct network layout
lib/climateEquipmentLayout.ts	Equipment 3D positioning
lib/climateEquipmentCapacity.ts	Capacity calculations
lib/cultivationLayout.ts	Crop grid layout
lib/cultivationFactors.ts	Crop-specific factors
lib/equipmentAwareHeatmap.ts	Equipment-influenced heatmap
lib/heatmapFallback.ts	Heatmap data fallback
lib/heatmapInfluence.ts	Per-system heatmap rules

lib/heatmapRevision.ts	Stable render revision token
lib/heatmapSurfaceUv.ts	Wall UV mapping
lib/plantGeometry.ts	Plant instance geometry
lib/previewMicroclimate.ts	Quick preview solver
lib/psychrometrics.ts	Basic psychrometrics
lib/shadingScreen.ts	Screen shading model
lib/structureUtils.ts	Structure geometry helpers
lib/thermalEstimate.ts	Thermal estimate wrapper
lib/climateExport.ts	Client-side export trigger
lib/apiConfig.ts	API base URL configuration

GAIA (AI Assistant)

File	Responsibility
lib/gaia/client.ts	GAIA API client
lib/gaia/buildContext.ts	Context assembly
lib/gaia/formatContext.ts	Context formatting
lib/gaia/prompts.ts	Prompt templates
lib/gaia/preprocessMarkdown.ts	Markdown preprocessing
lib/gaia/exportChat.ts	Chat export utility

i18n

Path	Responsibility
i18n.ts	react-i18next initialization
locales/{en,it,es,fr}/*.json	Translation namespaces (5 per locale)

Database

File	Responsibility
supabase/migrations/001_initial_schema.sql	profiles, greenhouses, RLS, triggers

Documentation

File	Responsibility
README.md	Project overview and quick start
CHANGELOG.md	Architectural change log
docs/ARCHITECTURE.md	System design reference
docs/API_SPEC.md	API specification
docs/THERMAL_EQUIPMENT_AUDIT.csv	Equipment coefficient audit
docs/ip-vault/	This IP knowledge vault

Continue to 12-IP-Declaration

Intellectual Property Declaration

Home · 00-IP-Deposit-Cover · 11-Module-Index

Ownership Statement

Software name: GreenhouseOS 5.0

Copyright holder: GreenhouseOS Contributors

Copyright year: 2026

License: Business Source License 1.1 (BUSL-1.1)

Change date: 2029-08-03 (converts to Apache License 2.0)

The authors declare that GreenhouseOS 5.0 is original software created by the GreenhouseOS Contributors. All architectural designs, algorithm implementations, user interface concepts, and documentation described in this vault are proprietary to the copyright holder, subject to the BUSL-1.1 license terms.

Originality Claims

The following elements represent original creative and technical work:

1. Unified Virtual Twin Architecture

Integration of real-time FAO-56 agronomic physics, quasi-steady-state thermal energy balance, equipment-aware spatial heatmaps, and industrial climate-computer export within a single browser-based 3D design environment. No prior open-source greenhouse design tool combines all four pillars (see 01-Executive-Summary) in one platform.

2. No-Form 3D Design Paradigm

Direct manipulation of greenhouse geometry via in-viewport TransformControls gizmos, eliminating traditional form-based configuration. Parameter changes propagate simultaneously to 3D mesh, thermal solver, and heatmap shader.

3. Frontend–Backend Thermal Parity

Deliberate synchronization of physical coefficients (ventilation ACH boosts, latent heat conversion, RH coupling, cooling capacity factor) between Python backend and TypeScript frontend solvers, enabling sub-50 ms client preview with server-validated results.

4. Equipment-Aware Spatial Heatmap

Custom GLSL shader system that visualizes per-cell temperature and VPD influenced by the spatial layout of climate equipment (pad walls, AC duct networks, HAF fans, ridge vents, heating pipes). The influence rules are documented in 04-Thermal-Simulation and docs/THERMAL_EQUIPMENT_AUDIT.csv.

5. Multi-AI Gateway with Deterministic Fallback

Decoupled provider abstraction supporting four AI backends with automatic fallback to a rule-based local optimizer that produces industrial climate-computer-compatible setpoints without cloud dependency.

6. Industrial Climate Computer Export

Vendor-specific JSON generation (Priva, Ridder, Hoogendoorn) from simulation-derived microclimate data, bridging agronomic modeling with operational greenhouse automation.

7. Multilingual No-Form UX

Complete internationalization (English, Italian, Spanish, French) across 5 namespaces, integrated with a 3D design workflow that has no traditional form fields.

Third-Party Components

GreenhouseOS incorporates the following open-source libraries under their respective licenses:

Library	License	Usage
FastAPI	MIT	Backend HTTP framework
Pydantic	MIT	Data validation
React	MIT	Frontend UI framework
Three.js	MIT	3D rendering
@react-three/fiber	MIT	React Three.js bindings
Zustand	MIT	State management
react-i18next	MIT	Internationalization
@supabase/supabase-js	MIT	Database client
Tailwind CSS	MIT	Utility-first CSS

Third-party components are used as libraries; the original architecture, physics integration, and application logic are proprietary.

Trade Secret Considerations

The following are considered trade secrets and are not disclosed in this vault:

- Exact calibration coefficients in THERMAL_EQUIPMENT_AUDIT.csv (referenced but not reproduced)
- AI system prompt full text (structure described, content withheld)
- Performance optimization techniques in the WebSocket handler
- Specific crop coefficient tuning values per cultivar

Deposit Scope

This vault documents:

- 113 source files (~15,231 lines of code)
- 6 completed development milestones
- 4 physics engines (FAO-56, VPD, DLI, thermal balance)
- 4 AI provider integrations + local fallback
- 3 industrial export formats
- 4 supported locales
- 1 database migration with RLS

Certification

This document was compiled on 2026-09-09 as a technical annex for intellectual property registration purposes. It contains no credentials, production infrastructure details, customer data, or full source code listings, in accordance with 10-Security-Redaction-Policy.

Return to Home